3520 Final Project

Julia Duffy

The sections I chose to implement were 1: ☆, 3: ☆☆, 5: ☆☆, 6: ☆☆ 8: ☆☆
Total (9 stars): ☆☆☆☆☆☆☆☆☆. All of the tests I added are at the bottom of each class. I also had to change any pre-existing tests that called parse and interp as I changed the declarations for those methods.

Section 5 - Local Bindings with 'Let':

The first part I chose to implement was Section 5. In Section 5, we were asked to implement local bindings with let expressions. I wanted to start with this section because I felt that it would involve the most structural changes out of all of the parts I was planning to complete. I ended up implementing an environment to store local bindings. This environment was added as a parameter of interp. I also introduced a new type **varE(x :: Symbol)** (which is basically idE). I considered storing fields in the environment as well but ultimately decided against that for encapsulation purposes. Instead, fields are accessed through the 'this' keyword or by looking them up in the object. This separation keeps the environment specifically for local variables introduced by let. I added tests to ensure that let expressions worked correctly in both simple cases and nested bindings. One interesting case was handling shadowing—when a local binding in an inner let block has the same name as a binding in an outer block. I verified that the interpreter correctly resolves to the inner binding in these cases. For example, the test: **let x = 10: let x = 20: x + x** produces a correct result of 40. This was handled by extending the environment upon a new let call.

```
type Binding
| bind(name :: Symbol, val :: Value)

type Env = Listof(Binding)

def mt_env = []
def extend_env = cons
```

Section 6 - Direct Field Access Without 'This' Keyword: For Section 6, we were asked to implement field access without the use of a "this" keyword. We were also asked to extend our interpreter to support method argument names othert than "arg". To do this, I ended up modifying parse to take in the method argument name. Any symbols passed into parse were compared to this argument name and if they matched up, I returned an argI() object for that case.

```
fun parse(s :: Syntax, arg_name :: Symbol) :: ExpI:
  cond
  | syntax_is_integer(s):
      intI(syntax_to_integer(s))
  | syntax_is_symbol(s):
      cond
      | syntax_to_symbol(s) == arg_name:
          argI()
      | syntax_to_symbol(s) == #'arg:
          argI()
      | syntax_to_symbol(s) == #'this:
          thisI()
      | syntax_to_symbol(s) == #'null:
          nullI()
      | ~else:
          varI(syntax_to_symbol(s))
```

Because interp later takes in a method argument value, I was able to match up that value to the **ArgI()** expression. As for being able to acces field without the "this" keyword, I just added some checks to **VarI()** in interp to first check if the variable is contained in the local environment and if not, it then checks to see if that variable represents a method argument. Ultimately, varI/varE were used to hold both method arguments and local variables.

```
  | varI(name):
      try:
        find(env, name)  // Check local variables first
        ~catch:
          // Check fields of the current class
          match this_type
          | objT(class_name):
              try:
                find_field_in_tree(name, class_name, t_classes)
                ~catch:
                   error(#'interp_i, "free variable")
          | ~else:
              error(#'interp_i, "free variable")
```

Section 3 - If/Then Statement: This was one of the easier sections for me to implement. I simply added the parsing steps that we have done in several assignments so far. The hardest part was implementing the typechecker. To do this, I had to add a helper function**, find_lca,** (find least common ancestor) which computes the least common ancestor of two types in the class hierarchy. This allowed the typechecker to determine the resulting type of the entire **ifl**() expression when the branches had different object types. The letl() case of the typechecker also checks to make sure that the branches of an if statement aren't an int and an object. It also makes sure that the condition is an int.

```
| ifI(cond_exp, then_exp, else_exp):
    let cond_type = recur(cond_exp, env):
        if cond_type == intT()
        | let then_type = recur(then_exp, env):
            match then_type:
            | intT():
                let else_type = recur(else_exp, env):
                  if else_type == intT()
                  | intT()
                  | error(#'typecheck, "type mismatch")
            | objT(class_name):
                let else_type = recur(else_exp, env):
                  objT(find_lca(objT.class_name(then_type), objT.class_name(else_type), t_classes))
        | error(#'typecheck, "Condition must evaluate to an integer")
```

Section 8 - Null: In Section 8, we were asked to introduce null. This involved adding support for null as both an expression and a type within the language. First, I extended the grammar to include a null expression, representing a null value, and updated the parser to recognize and properly handle null. From there, I added typechecking logic to ensure that null could be assigned to any object type, but not to primitive types like intT(). In the typecheck_exp function, I added a case for nullI that returned the type **objT(#'Object)** by default, since null is considered a subtype of all object types. This allowed null to be used in contexts where an object type was expected. I also updated the typechecking for constructs like **getI** and **sendI** to raise errors if null was accessed.

Section 1 - Type Casting: I decided to do section 1 with the hopes of getting some extra credit. This section wasn't too involved. The parsing and interpreting steps were fairly straight forward. In interp, I check to see if the types of the casting expression are valid. In the typechecker, I

check to see of the type being casted is a subtype of the class in the parentheses. This also wasn’t too difficult as we were already provided an is_subclass method.